

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

COURSE NAME: Deep Learning **COURSE CODE:** MLA0403

Faculty: Dr. Arul Raj A.M

Name of the Student: K. Vannur Swamy

Reg. No: 192525265

GitHub Link: _____

CO1 Weightage: 15% **Assessment Tool Weightage:** 35% **Marks:** 50

1. Learning Algorithms – Linear Regression using Gradient Descent

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1,2,3,4,5], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)
m, c = 0.0, 0.0
lr = 0.01
losses = []
for i in range(1000):
    pred = m*X + c
    error = pred - y
    loss = np.mean(error**2)
    losses.append(loss)
    m -= lr * np.mean(error*X)
    c -= lr * np.mean(error)
print("m =", m, "c =", c)
plt.plot(losses)
plt.xlabel("Iterations"); plt.ylabel("Loss"); plt.title("Learning Curve")
plt.show()
```

Result: The loss decreases continuously and the model learns approximately $y = 2x$.

2. Maximum Likelihood Estimation (MLE)

```
import numpy as np

np.random.seed(1)
actual_mean, actual_var = 5, 4
data = np.random.normal(actual_mean, np.sqrt(actual_var), 1000)

mle_mean = np.mean(data)
mle_var = np.mean((data - mle_mean)**2)
print("Actual mean:", actual_mean)
print("MLE mean:", mle_mean)
print("Actual variance:", actual_var)
print("MLE variance:", mle_var)
```

Result: The estimated mean and variance are close to the actual parameters.

3. Building Machine Learning Algorithms

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```

model = LogisticRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))

```

Result: The classification model gives high accuracy and the confusion matrix shows correct and incorrect predictions.

4. Neural Networks

```

import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
model = MLPClassifier(hidden_layer_sizes=(10,), activation='relu',
                      max_iter=2000, random_state=42)
model.fit(Xtr, ytr)
print("Accuracy:", accuracy_score(yte, model.predict(Xte)))

```

Result: The hidden layer learns the non-linear boundary, so the network can classify the curved pattern.

5. Artificial Neurons – Sigmoid and ReLU

```

import numpy as np

x = np.array([-2, -1, 0, 1, 2])
sigmoid = 1 / (1 + np.exp(-x))
relu = np.maximum(0, x)
print("Input      :", x)
print("Sigmoid    :", sigmoid)
print("ReLU       :", relu)

```

Result: Sigmoid produces values between 0 and 1, while ReLU outputs 0 for negative inputs and the input itself for positive values.

6(a). Perceptron Algorithm

```

from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification, make_circles
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=100, n_features=2,
                          n_redundant=0, n_informative=2,
                          n_clusters_per_class=1, random_state=42)

p = Perceptron(max_iter=1000, random_state=42)
p.fit(X, y)
print("Linearly separable accuracy:", accuracy_score(y, p.predict(X)))

X2, y2 = make_circles(n_samples=100, factor=0.5, noise=0.05, random_state=42)
p2 = Perceptron(max_iter=1000, random_state=42)
p2.fit(X2, y2)
print("Non-linear accuracy:", accuracy_score(y2, p2.predict(X2)))

```

Result: Perceptron works well for linearly separable data but cannot properly learn non-linear boundaries.

6(b). Multilayer Perceptron (MLP) vs Single-Layer Perceptron

```

from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_moons
from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=200, noise=0.1, random_state=42)

p = Perceptron(max_iter=2000, random_state=42)
p.fit(X, y)

mlp = MLPClassifier(hidden_layer_sizes=(10,), max_iter=2000, random_state=42)
mlp.fit(X, y)
print("Perceptron:", accuracy_score(y, p.predict(X)))
print("MLP:", accuracy_score(y, mlp.predict(X)))

```

Result: MLP generally performs better on non-linear data because hidden layers learn complex patterns.

7(a). Forward Propagation

```
import numpy as np

x = np.array([1.0, 2.0])
W1 = np.array([[0.5, 0.2], [0.4, 0.3]])
b1 = np.array([0.1, 0.1])
W2 = np.array([0.6, 0.7])
b2 = 0.2

def sigmoid(z):
    return 1/(1+np.exp(-z))

h = sigmoid(W1 @ x + b1)
out = sigmoid(W2 @ h + b2)
print("Hidden output:", h)
print("Final output:", out)
```

Result: Forward propagation computes hidden-layer activations first and then the final network output.

7(b). Backpropagation

```
import numpy as np

x = np.array([1.0, 2.0])
y = 1.0
w = np.array([0.2, 0.4])
b = 0.1
lr = 0.1

z = np.dot(w, x) + b
pred = 1/(1+np.exp(-z))
error = pred - y

grad_w = error * pred * (1-pred) * x
grad_b = error * pred * (1-pred)

w -= lr * grad_w
b -= lr * grad_b
print("Updated weights:", w)
print("Updated bias:", b)
```

Result: Backpropagation calculates gradients and updates weights in the direction that reduces the loss.

8(a). Gradient Descent and Learning Rate

```
import numpy as np
def gradient_descent(lr, steps=30):
    x = 10.0
    values = []
    for _ in range(steps):
        values.append(x*x)
        x -= lr * 2*x
    return values
for lr in [0.01, 0.1, 0.5]:
    v = gradient_descent(lr)
    print(lr, "final cost =", v[-1])
```

Result: A suitable learning rate gives fast convergence. A very small rate is slow, while a very large rate can overshoot or become unstable.

8(b). Stochastic Gradient Descent (SGD)

```
import numpy as np

X = np.array([1,2,3,4,5], dtype=float)
y = np.array([2,4,6,8,10], dtype=float)
w, b, lr = 0.0, 0.0, 0.01

for epoch in range(100):
    for xi, yi in zip(X, y):
        pred = w*xi + b
        error = pred - yi
        w -= lr * 2*error*xi
        b -= lr * 2*error

print("w =", w, "b =", b)
```

Result: SGD updates parameters after each training sample. It usually converges faster initially but has more fluctuations than batch Gradient Descent.

9. Mini-Batch Gradient Descent

```
import numpy as np

X = np.array([1,2,3,4,5,6,7,8], dtype=float)
y = 2*X
w, b, lr = 0.0, 0.0, 0.01
batch_size = 2
for epoch in range(100):
    for i in range(0, len(X), batch_size):
        xb = X[i:i+batch_size]
        yb = y[i:i+batch_size]
        error = w*xb + b - yb
        w -= lr * 2*np.mean(error*xb)
        b -= lr * 2*np.mean(error)

print("w =", w, "b =", b)
```

Result: Mini-batch training provides a balance between batch GD stability and SGD speed. Batch size affects training stability and computational efficiency.

10. Curse of Dimensionality

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
for d in [2, 10, 50, 100]:
    X, y = make_classification(n_samples=500, n_features=d,
                              n_informative=min(2,d), random_state=42)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(Xtr, ytr)
    print("Dimensions:", d, "Accuracy:", accuracy_score(yte, model.predict(Xte)))
```

Result: As dimensionality increases, data points become relatively farther apart and distance-based models can become less effective. This is called the curse of dimensionality.

Student Name: K. Vannur Swamy **Reg. No:** 192525265